

HW3: High-Performance MPI Circle Renderer Technical Report

Team 18

April 30, 2026

1 Introduction

In this project, we implemented a parallelized circle renderer using MPI. The goal was to optimize the rasterization of large circle datasets on a distributed-memory cluster (`rdma0-rdma3`, 4 nodes). We adopted a domain decomposition strategy combined with a back-to-front rendering optimization that exploits opaque overwrite semantics, and conducted three complementary studies: (1) serial-baseline runtime versus input size, (2) strong scaling on the 1M-circle input, and (3) weak scaling at constant 125k circles per process.

2 Implementation Details

2.1 Parallelization Strategy: Strip Rendering

We implemented **Domain Decomposition** (Strip Rendering) to parallelize the workload. The image is partitioned into horizontal row bands; each rank is assigned a contiguous range `[my_start, my_end)` computed from the global image height H and the process count.

- **Quick Reject:** Each rank iterates through all circle records but tests the bounding box `[ymin, ymax]` against its own row range. Circles whose bounding boxes lie entirely outside the local strip are skipped immediately, so processes assigned to empty regions waste no compute.
- **Memory Footprint:** We render directly into an `unsigned char` RGB buffer, reducing pixel storage from 12 bytes (`float`) to 3 bytes per pixel.

2.2 Back-to-Front Rendering with Per-Pixel Mask

The key algorithmic optimization in our method comes from observing that opaque overwrite means *the last circle covering a pixel wins*. By walking the circle list in **reverse order** and only painting pixels that have not yet been painted, we obtain two layers of early exit:

- **Per-row skip:** A counter `unpainted[local_y]` tracks remaining unpainted pixels per row; when it reaches zero, the entire row is skipped for all subsequent (i.e., lower) circles.
- **Global early exit:** A scalar `total_unpainted` aggregates the per-row counters; once the entire strip is fully painted, the outer loop terminates immediately.

For dense scenes with heavy overlap, most circles in the lower layers are never touched, dramatically reducing per-rank compute. As shown in Section 4, this is so effective that on the 1M-circle input the parallel renderer becomes communication-bound at very small process counts.

2.3 Communication Pattern

Communication is handled with two collective calls:

- `MPI_Bcast` distributes the full record buffer ($1,000,000 \text{ records} \times 15 \text{ B} \approx 15 \text{ MB}$) from rank 0 to all ranks. To avoid the `int` count limit, the broadcast is chunked in 1 GiB segments.
- `MPI_Gatherv` collects each rank’s pre-rendered strip into the final framebuffer on rank 0. Because each rank produces a strip that fits exactly into its slice of the global image, no compositing logic is needed.

2.4 Hardware Device Adaptation

The specified InfiniBand/RoCE device `mlx5_0:1` was found to be mapped to `rocep23s0:1` on the Ubuntu 24.04 nodes. By explicitly setting:

```
UCX_TLS=rc,sm,self UCX_NET_DEVICES=rocep23s0:1
```

we successfully leveraged the 100 Gbps RoCE interconnect via UCX (`rc` for inter-node, `sm` for intra-node shared memory, `self` for loopback).

2.5 Execution Command and Timing

All MPI experiments were launched with:

```
UCX_TLS=rc,sm,self UCX_NET_DEVICES=rocep23s0:1 UCX_LOG_LEVEL=warn \
mpirun --hostfile hosts -npernode <NPP> --mca pml ucx --mca btl ^
tcp \
./renderer_mpi <input.bin> <output.png>
```

The serial baseline `renderer` reports *Total render time*, which spans the serial render loop through PNG write. The MPI program reports rank 0’s *Total time*, which includes the root file read, `MPI_Bcast`, strip rendering, `MPI_Gatherv`, and PNG write. Each configuration was repeated 3 times. Because multi-node UCX runs occasionally exhibit cold-start jitter on first endpoint setup, we report the full trial set but use the *minimum* for the headline analysis.

3 Serial Baseline and Input Sizes

Table 1 lists the five input files together with their image size, file size, and the three serial trials. The 1M-circle dataset `large_c1000000.bin` is used for strong scaling; the three `large_c{1,2,4}M` datasets share the same 640×480 image and form the weak-scaling sequence.

Input	Circles	Image	File	Serial trials (s)	min (s)	median (s)
<code>imbalance_c100000.bin</code>	100,000	320×240	1.5 MB	0.555, 0.584, 0.606	0.555	0.584
<code>medium_c200000.bin</code>	200,000	640×480	3.0 MB	0.804, 0.812, 0.827	0.804	0.812
<code>large_c1000000.bin</code>	1,000,000	640×480	15.0 MB	2.595, 2.719, 2.732	2.595	2.719
<code>large_c2000000.bin</code>	2,000,000	640×480	30.0 MB	4.713, 4.759, 5.063	4.713	4.759
<code>large_c4000000.bin</code>	4,000,000	640×480	60.0 MB	8.364, 8.483, 8.725	8.364	8.483

Table 1: Input datasets and serial baseline timing.

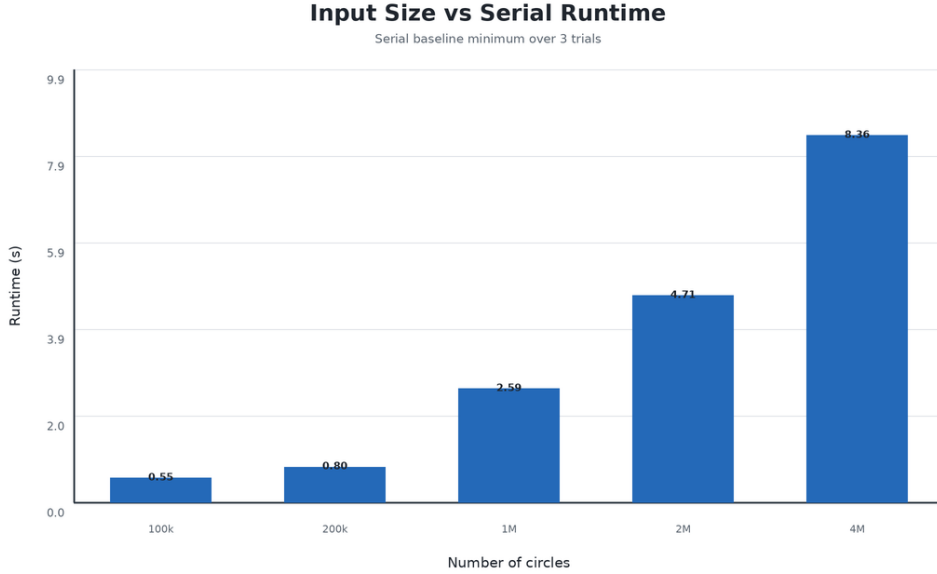


Figure 1: Serial baseline runtime versus circle count.

For the three large inputs, the image is fixed at 640×480 , so runtime is dominated by circle count. From 1M to 4M circles, serial-min grows from 2.595s to 8.364s — close to linear in circle count, though not perfectly so because circle radius distribution, cache behavior, and PNG output cost also contribute.

4 Strong Scaling

Strong scaling fixes the problem at `large_c1000000.bin` (1M circles, 640×480) and varies (N, NPP) for nodes $N \in \{1, 2, 3, 4\}$ and per-node processes $\text{NPP} \in \{2, 4, 8, 16, 32\}$. Speedup and efficiency are computed against the serial baseline minimum:

$$\text{Speedup} = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{2.594543 \text{ s}}{T_{\text{parallel}}}, \quad \text{Efficiency} = \frac{\text{Speedup}}{P_{\text{total}}}.$$

Caveat on efficiency > 1. `renderer_mpi` is not just a parallel port of `renderer`; it also incorporates the back-to-front opaque-overwrite algorithm with per-row and global early exit. Speedup against the serial baseline therefore reflects *both* the algorithmic improvement and parallel execution, which is why low-process configurations show super-linear efficiency. This is an honest accounting of end-to-end speedup, not a parallel-efficiency anomaly.

4.1 Full Strong-Scaling Table

Table 2 reports all 60 trials, the per-config min, speedup, and efficiency. Bold values are clear cold-start outliers, attributed to UCX establishing RC connections on a fresh process map.

Nodes	NPP	Total P	Trials (s)	min (s)	Speedup	Efficiency
1	2	2	0.143, 0.204, 0.207	0.143	18.14×	9.07
1	4	4	0.169, 0.171, 0.174	0.169	15.39×	3.85
1	8	8	0.149, 0.153, 0.154	0.149	17.39×	2.17
1	16	16	0.161, 0.163, 0.164	0.161	16.08×	1.00
1	32	32	0.163, 0.164, 0.164	0.163	15.89×	0.50
2	2	4	0.189, 0.191, 0.195	0.189	13.75×	3.44
2	4	8	0.161, 0.166, 1.930	0.161	16.10×	2.01
2	8	16	0.181, 1.632 , 2.100	0.181	14.35×	0.90
2	16	32	0.253, 0.254, 0.272	0.253	10.25×	0.32
2	32	64	0.292, 0.297, 0.315	0.292	8.87×	0.14
3	2	6	0.225, 0.226, 0.228	0.225	11.53×	1.92
3	4	12	0.178, 0.182, 1.674	0.178	14.58×	1.22
3	8	24	0.195, 0.196, 0.197	0.195	13.30×	0.55
3	16	48	0.263, 0.263, 0.269	0.263	9.87×	0.21
3	32	96	0.351, 0.360, 0.362	0.351	7.38×	0.08
4	2	8	0.175, 1.816 , 3.447	0.175	14.79×	1.85
4	4	16	0.200, 0.220, 0.226	0.200	13.00×	0.81
4	8	32	0.262, 0.267, 0.271	0.262	9.89×	0.31
4	16	64	0.308, 0.310, 0.320	0.308	8.41×	0.13
4	32	128	0.358, 0.359, 0.386	0.358	7.25×	0.06

Table 2: Strong-scaling results on `large_c1000000.bin`. Bold values mark cold-start outliers.

4.2 Strong-Scaling Plots

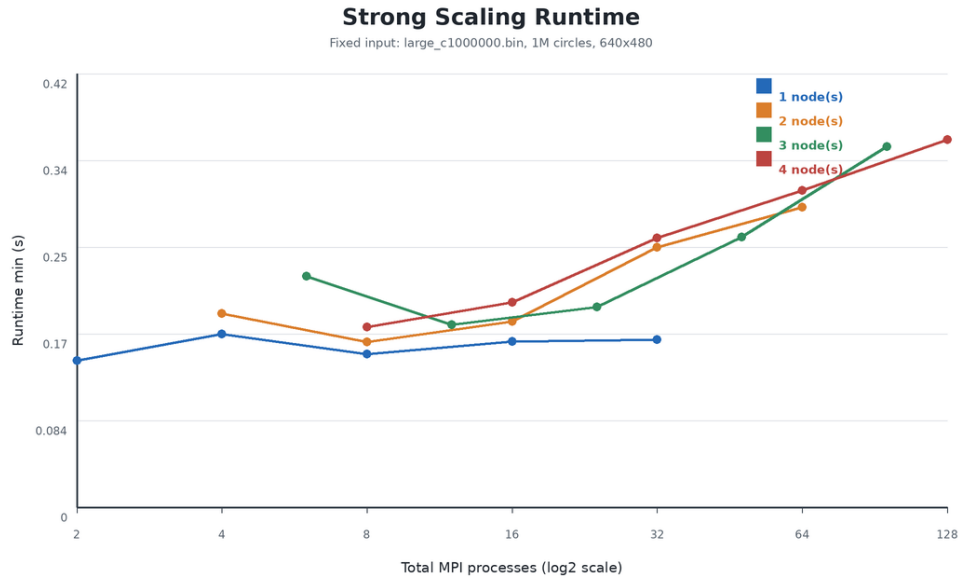


Figure 2: Strong-scaling runtime (min of 3 trials) versus total MPI processes, grouped by node count.

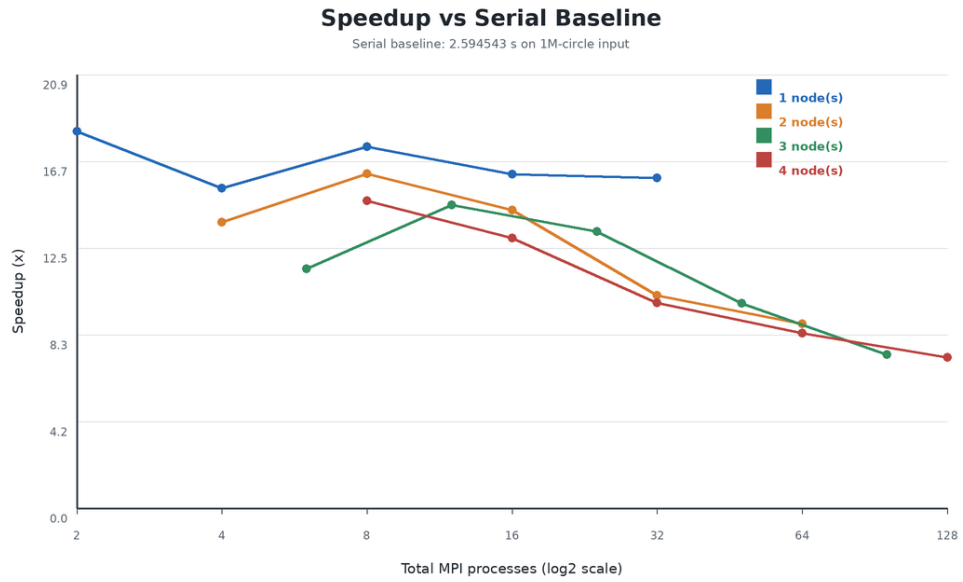


Figure 3: Speedup relative to the serial baseline (2.594543s).

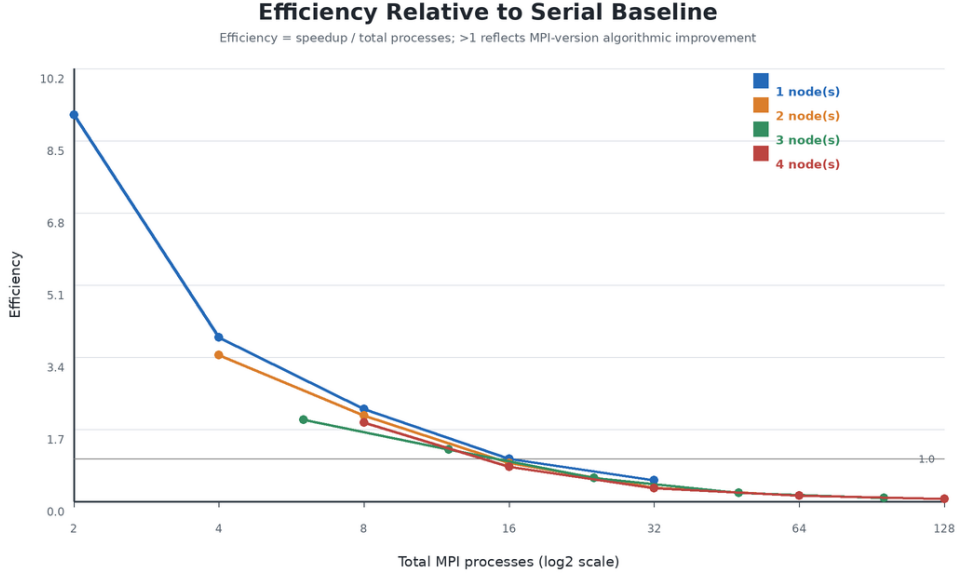


Figure 4: Efficiency relative to the serial baseline. Values > 1 reflect our algorithmic improvement folded into end-to-end speedup.

4.3 Strong-Scaling Observations

For this input size, adding more MPI processes does not consistently reduce runtime. The best measured time is 0.143s at 1 node $\times$ 2 processes (**18.14 $\times$** speedup). Many larger configurations are slower, especially at high process counts or across multiple nodes (Figure 2).

The cause is that our method’s algorithmic optimization leaves very little compute on a 1M-circle input, so fixed costs dominate:

- Rank 0 reads the entire record file.
- The full record array is broadcast to every rank.
- Each rank renders a strip; rank 0 then gathers all strips with `Gatherv`.
- Multi-node runs additionally pay UCX endpoint and RDMA setup overhead.

At high process counts, useful per-rank compute shrinks while broadcast/gather and process-management overhead remain — producing the reverse-scaling region visible in Figures 2 and 3.

5 Weak Scaling

Weak scaling holds the per-process workload constant at 125,000 circles while increasing both total processes and total circles, all on the 4-node hostfile:

- 4 nodes $\times$ 2 NPP = 8 processes, 1M circles
- 4 nodes $\times$ 4 NPP = 16 processes, 2M circles

Input	P	NPP	Circ./P	Trials (s)	min	med	mean	Eff.
large_c1000000.bin	8	2	125k	0.179, 0.180, 1.697	0.179	0.180	0.685	1.00
large_c2000000.bin	16	4	125k	0.292, 2.067, 2.323	0.292	2.067	1.561	0.61
large_c4000000.bin	32	8	125k	0.497, 0.502, 0.586	0.497	0.502	0.528	0.36

Table 3: Weak-scaling results: 4 nodes, constant 125,000 circles per process.

- 4 nodes $\times$ 8 NPP = 32 processes, 4M circles

Weak efficiency is T_8/T_P , normalized to the 8-process minimum.

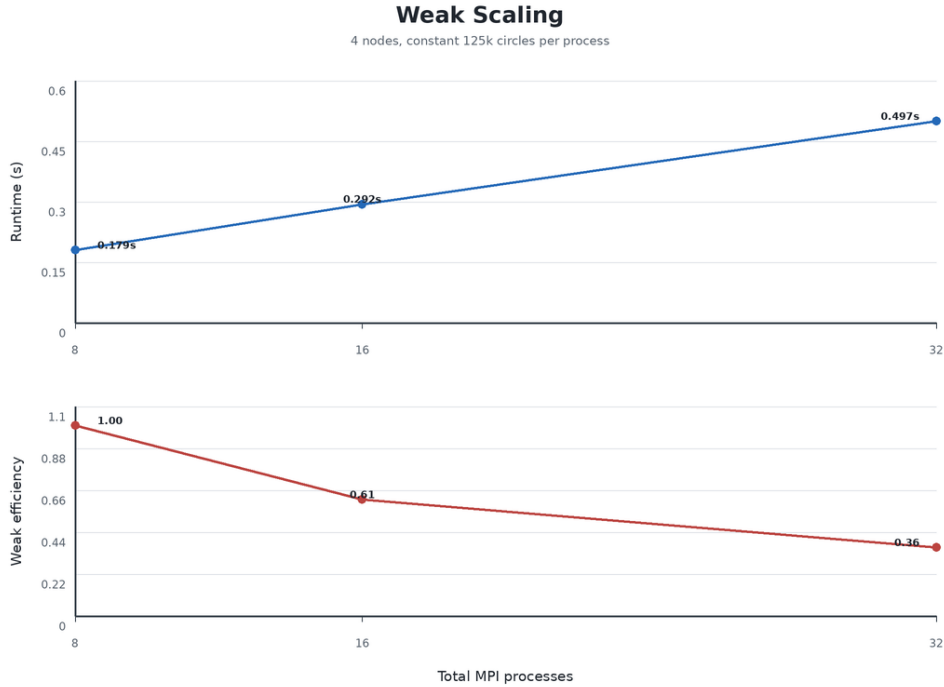


Figure 5: Weak-scaling runtime (top) and weak efficiency (bottom).

5.1 Weak-Scaling Observations

Ideal weak scaling would keep runtime nearly constant, but min runtime grows from 0.179s (8 P) to 0.497s (32 P) even though circles per process are fixed. Weak efficiency drops to 0.61 at 16 P and 0.36 at 32 P. This is dominated by overhead that scales with the *total* problem size rather than the local work:

- **Bcast** volume grows from 15 MB to 60 MB — every rank receives the entire circle array.
- More ranks participate in collectives, increasing latency and synchronization cost.
- **Gatherv** still funnels every rendered strip back to rank 0.
- Cold-start jitter is visible in the 16 P trial (median 2.067s vs. min 0.292s).

6 Conclusion

The MPI implementation achieves its best measured performance on the 1M-circle input at 1 node $\times$ 2 processes per node, with a runtime of 0.143s and a speedup of **18.14 $\times$** over the serial baseline. However, increasing the number of MPI processes does not continue to improve performance. After small process counts, strong scaling degrades because our optimized renderer finishes the computation quickly, so broadcast, gather, and multi-node setup overhead become the dominant costs.

The weak-scaling results show the same limitation. Even with a constant 125,000 circles per process, runtime increases from 0.179s at 8 processes to 0.497s at 32 processes, and weak efficiency drops to 0.36. This indicates that the implementation is communication-bound at larger process counts. The main reason is that every rank receives the full circle dataset, so communication grows with total input size rather than staying proportional to each rank’s local work.

Overall, `renderer_mpi` is highly effective compared with the serial baseline, mainly because of its back-to-front opaque overwrite strategy and early stopping. For the measured inputs, the best configuration is a small process count; larger multi-node configurations introduce more communication overhead than useful parallel speedup.